

Greedy Algorithm (DSA) - Complete Notes

Definition

A Greedy Algorithm chooses the best possible local choice at every step to reach an optimal solution. It never revisits previous decisions. It works when the problem satisfies the Greedy Choice Property and Optimal Substructure.

General Algorithm

- Start
- Choose the best available option
- Add it to the solution
- Remove it from future consideration
- Repeat until finished
- End

Real-Life Example: ATM Coin Change

Amount = ₹760, Notes = {₹500, ₹200, ₹50, ₹10}

Step	Note	Remaining
1	500	260
2	200	60
3	50	10
4	10	0

Result: $500 + 200 + 50 + 10 = ₹760$

Advantages

- ✓ Simple to implement
- ✓ Fast execution
- ✓ Low memory usage
- ✓ Efficient for optimization problems

Limitations

- ✗ May not produce global optimum
- ✗ Cannot backtrack
- ✗ Works only for specific problems

Common Interview Problems

- Activity Selection
- Fractional Knapsack
- Coin Change
- Huffman Coding

- Job Sequencing
- Dijkstra Algorithm
- Prim's Algorithm
- Kruskal's Algorithm
- Minimum Platforms
- Connecting Ropes

Greedy vs Dynamic Programming

Greedy	Dynamic Programming
Local optimum	Global optimum
No backtracking	Stores subproblem results
Usually faster	More computation

Simple Greedy Program: Coin Change

Problem: Find minimum coins for Amount=28 using {10,5,2,1}.

Sample Input: 28

Sample Output: 10, 10, 5, 2, 1 (Total Coins = 5)

Algorithm:

- Read amount.
- Store coins in descending order.
- For each coin, while amount $\geq$ coin: print coin, subtract coin, increment count.
- Print total count.

```
int[] coins={10,5,2,1};
int amount=28,count=0;
for(int coin:coins){
    while(amount>=coin){
        System.out.println(coin);
        amount-=coin; count++;
    }
}
System.out.println("Total Coins="+count);
```

Time Complexity: $O(n)$ where n is the number of coin denominations. Space Complexity: $O(1)$.